UQAR

TECHNOLOGIES DE L'INFOROUTE

INF37407 (MS)

UNIVERSITÉ DU QUÉBEC À RIMOUSKI - CAMPUS LÉVIS

DÉPARTEMENT DE MATHÉMATIQUES, D'INFORMATIQUE ET DE GÉNIE

RAPPORT

Travail Pratique 1 : TravelHub

ÉQUIPE :
Ibrahim Soumaila Cissé
Johan Mintsa Mi-Toung

PROFESSEUR :
Yacine Yaddaden, Ph. D.

Date : 29 octobre 2026

Table des matières

1 Introduction **5**
1.1 Contexte 5
1.2 Objectifs 5
1.3 Périmètre 5
1.4 Organisation du rapport 5

2 Environnement et technologies **6**
2.1 Versions retenues 6
2.2 Pourquoi Django 5.2 LTS 6
2.3 Configuration 7

3 Architecture **7**
3.1 Applications Django 7
3.2 Arborescence 8
3.3 Qui fait quoi 8
3.4 Parcours d'une requête 8

4 Modèle de données **8**
4.1 Diagramme entité-association 8
4.2 Relations 10
4.3 Utilisateur personnalisé 12
4.4 Contraintes d'intégrité 12
4.5 Migrations 14

5 Gestion des voyages, destinations, activités et participants **14**
5.1 QuerySets réutilisables 14
5.2 Relations inverses et préchargement 15
5.3 Agrégations 16
5.4 Services métier 16
5.5 Administration et données de démonstration 17

6 Intégrations externes **17**
6.1 Open-Meteo (REST) 17
6.2 Countries (GraphQL) 19
6.3 Couche HTTP commune 19
6.4 Cache 20
6.5 Agrégation et dégradation partielle 20

7 API REST **21**
7.1 Points d'accès 21
7.2 Serializers 23

7.3 ViewSets et routage 23
7.4 Filtres, recherche, tri et pagination 24
7.5 Codes HTTP et format des erreurs 25
7.6 Documentation OpenAPI 26

8 API GraphQL **26**
8.1 Schéma 26
8.2 Réutilisation de la logique REST 27
8.3 Authentification, limites et erreurs 28
8.4 Durcissement et performance 28
8.5 Exemple 29
8.6 Comparaison REST et GraphQL 29

9 Validation, transactions et intégrité **31**
9.1 Trois niveaux de validation 31
9.2 Appliquer `Model.clean()` depuis DRF 32
9.3 Contraintes violées malgré la validation 32
9.4 Transactions 33
9.5 Démonstration du rollback 33

10 Sécurité **34**
10.1 Authentification par JWT 34
10.2 Permissions par rôle 34
10.3 Contrôle d'accès au niveau objet 35
10.4 Configuration de production 35
10.5 OWASP API Security Top 10 35
10.6 Risques résiduels 37

11 Tests **37**
11.1 Stratégie 37
11.2 Couverture fonctionnelle 37
11.3 Résultats et couverture 37
11.4 Collection Postman 37
11.5 Qualité du code 39

12 Industrialisation **39**
12.1 Docker 41
12.2 Intégration continue 41
12.3 Journaux et contrôle de santé 41
12.4 Sauvegarde et restauration 42

13 Conclusion **42**
13.1 Bilan 42
13.2 Limites 43

1 Introduction

1.1 Contexte

TravelHub est une plateforme qui sert à planifier des voyages à plusieurs. Un utilisateur crée un voyage et le découpe en destinations. Il prévoit ensuite des activités à chaque étape. Il peut aussi partager son voyage avec d'autres utilisateurs en leur donnant un rôle, qui peut être propriétaire, éditeur ou lecteur. Pour chaque destination, l'application va chercher la météo du séjour et quelques informations sur le pays.
Dans ce rapport, nous présentons le back-end que nous avons réalisé pour le premier travail pratique. Nous expliquons nos choix techniques, l'architecture, le modèle de données et les deux API. Nous parlons aussi de l'intégration des services externes, de la sécurité et des tests. Tout ce qui est décrit ici se retrouve dans le code du dépôt.

1.2 Objectifs

Nous nous sommes fixé six objectifs :

- modéliser les données dans PostgreSQL avec des relations 1-N et N-N, et laisser la base vérifier elle-même les contraintes d'intégrité ;
- offrir les mêmes fonctions avec une API REST (Django REST Framework) et avec une API GraphQL (Graphene-Django) ;
- utiliser deux API publiques, Open-Meteo (REST) et Countries (GraphQL) ;
- protéger l'accès avec des jetons JWT et des permissions basées sur le rôle, en suivant l'OWASP API Security Top 10 [13] ;
- garder des données cohérentes grâce à plusieurs niveaux de validation et aux transactions ;
- tout vérifier avec des tests automatisés et une collection Postman.

1.3 Périmètre

Le TP1 porte seulement sur le back-end, l'interface utilisateur viendra au TP2. Pour l'instant, l'API est donc la seule façon d'utiliser le système. On peut la tester avec Swagger UI, GraphiQL, Postman et l'administration de Django. Nous avons aussi ajouté une image Docker, de l'intégration continue, des journaux structurés et des sauvegardes. L'énoncé ne le demandait pas, mais cela nous servira pour déployer le TP2.

1.4 Organisation du rapport

La section 2 explique les versions que nous avons choisies. Les sections 3 et 4 présentent l'architecture et le modèle de données, puis la section 5 décrit la logique métier. La section 6 parle des services externes. Les deux API sont présentées et comparées aux

sections 7 et 8. Les sections 9 à 11 traitent de l'intégrité, de la sécurité et des tests. Enfin, la section 12 porte sur le déploiement et la section 13 conclut le rapport.

2 Environnement et technologies

2.1 Versions retenues

Le tableau 1 donne les versions fixées dans `requirements.txt` et `requirements-dev.txt`. Chaque dépendance a une version exacte. Deux installations donnent donc le même environnement.

TABLE 1 – Versions des outils et des bibliothèques.

Composant	Version	Rôle
Python	3.14	Langage
PostgreSQL	18	Base de données relationnelle [14]
Django	5.2.18 (LTS)	Cadriciel web, ORM, administration [4]
Django REST Framework	3.18.3	API REST [2]
SimpleJWT	5.5.1	Jetons JWT et liste noire des jetons révoqués [9]
drf-spectacular	0.30.0	Schéma OpenAPI 3, Swagger UI, ReDoc [5]
Graphene-Django	3.2.3	API GraphQL [7]
django-filter	26.2	Filtres des listes
django-cors-headers	4.9.0	Politique CORS
django-environ	0.14.0	Configuration par variables d'environnement
psycopg	3.3.6	Pilote PostgreSQL
requests	2.34.2	Appels HTTP vers les services externes
Gunicorn, WhiteNoise	26.2.0, 6.12.0	Serveur WSGI et fichiers statiques en production
pytest, pytest-django	9.1.1, 4.14.0	Tests
pytest-cov	7.1.0	Couverture de code
responses	0.26.3	Simulation des appels HTTP externes
Ruff	0.17.2	Analyse statique et formatage

Nous avons développé sous Windows 11. Les scripts d'exploitation sont donc en PowerShell, et nous avons géré la base avec pgAdmin 4.

2.2 Pourquoi Django 5.2 LTS

Nous voulions la dernière version stable de Django qui marche avec *toutes* nos dépendances. Django 6.1 était déjà sorti, mais Graphene-Django 3.2.3 et SimpleJWT 5.5.1 ne disent pas encore supporter Django 6. Django 5.2 est une version LTS, maintenue jusqu'en avril 2028, et elle supporte officiellement Python 3.14. Nous avons préféré un ensemble stable à la version la plus récente du cadriciel.

2.3 Configuration

Les paramètres sont répartis dans quatre modules (tableau 2). Les valeurs sensibles ne sont jamais écrites dans le code. `django-environ` les lit dans l'environnement, sinon dans un fichier `.env` que Git ignore. Toutes les variables sont expliquées dans `.env.example`.

TABLE 2 – Modules de paramètres.

Module	Usage
`config.settings.base`	Paramètres communs (applications, DRF, JWT, GraphQL, journaux)
`config.settings.dev`	Développement, avec `DEBUG`, GraphiQL et l'introspection activés
`config.settings.test`	Tests, avec un hachage rapide des mots de passe, un cache en mémoire et les journaux sur la console
`config.settings.prod`	Production, avec HTTPS, HSTS, cookies sécurisés, JSON seulement et WhiteNoise

3 Architecture

3.1 Applications Django

Nous avons suivi la structure conseillée par Django. Il y a un projet de configuration (`config`) et plusieurs applications rangées dans `apps/`. Chaque application s'occupe d'une seule chose (tableau 3).

TABLE 3 – Applications du projet.

Application	Responsabilité
`config`	Paramètres, routage principal, schéma GraphQL racine, points d'entrée WSGI et ASGI
`apps.core`	Éléments communs (`TimeStampedModel`, permissions par rôle, gestionnaire d'exceptions, vue GraphQL, journaux, contrôle de santé)
`apps.accounts`	Utilisateur personnalisé, inscription, jetons JWT, profil
`apps.trips`	Voyages, participants et services métier (création, clonage, statistiques)
`apps.itinerary`	Destinations, activités et coordonnées géographiques
`apps.integrations`	Clients Open-Meteo et Countries, couche HTTP commune, agrégateur

Les applications métier ont toutes les mêmes fichiers : `models.py`, `serializers.py`, `views.py`, `urls.py`, `filters.py`, `schema.py` (pour GraphQL), `admin.py`, `services.py` et un dossier `tests/`. Comme tout est rangé de la même façon, on s'y retrouve vite.

3.2 Arborescence

Extrait 1 – Arborescence du dépôt (simplifiée).

```
TravelHub/
|-- config/              settings/ (base, dev, test, prod), urls.py, schema.py
|-- apps/
|   |-- core/            permissions, exceptions, vue GraphQL, journaux
|   |-- accounts/        User, inscription, JWT
|   |-- trips/           Trip, TripParticipant, services
|   |-- itinerary/       Destination, Activity
|   `-- integrations/    services/ (http, open_meteo, countries, aggregator)
|-- postman/             collection v2.1 et environnement
|-- scripts/             backup.ps1, restore.ps1, db/
|-- .github/workflows/   ci.yml
|-- Dockerfile  docker-compose.yml  gunicorn.conf.py
`-- requirements.txt  requirements-dev.txt  pytest.ini  .env.example
```

3.3 Qui fait quoi

Nous avons essayé de respecter quelques règles simples.
Les vues restent courtes. Une vue authentifie l'utilisateur, choisit le queryset et vérifie la permission, puis elle passe la main. Elle ne contient pas de règle métier.
Les opérations qui touchent plusieurs tables (`create_trip`, `clone_trip`, `add_participant`) sont dans `services.py`. L'API REST et l'API GraphQL appellent les mêmes services.
Les règles de dates sont écrites dans `Model.clean()`, et les règles les plus importantes sont aussi vérifiées par des contraintes PostgreSQL.
Enfin, toutes les erreurs passent par un seul gestionnaire (`apps/core/exceptions.py`). Elles ont donc toutes la même forme JSON.

3.4 Parcours d'une requête

La figure 1 montre ce qui se passe pour une requête `PATCH /api/trips/12/`, du début à la fin.
L'ordre de ces étapes est important pour la sécurité. Le filtrage par `visible_to()` se fait avant la permission objet. Pour l'utilisateur, le voyage d'un autre n'existe donc pas. Il reçoit un 404 et pas un 403, ce qui ne lui apprend pas que la ressource existe.

4 Modèle de données

4.1 Diagramme entité-association

Le modèle a cinq tables métier (figure 2). Nous n'y avons pas mis les tables techniques de Django et de SimpleJWT (sessions, permissions, liste noire des jetons). Les champs

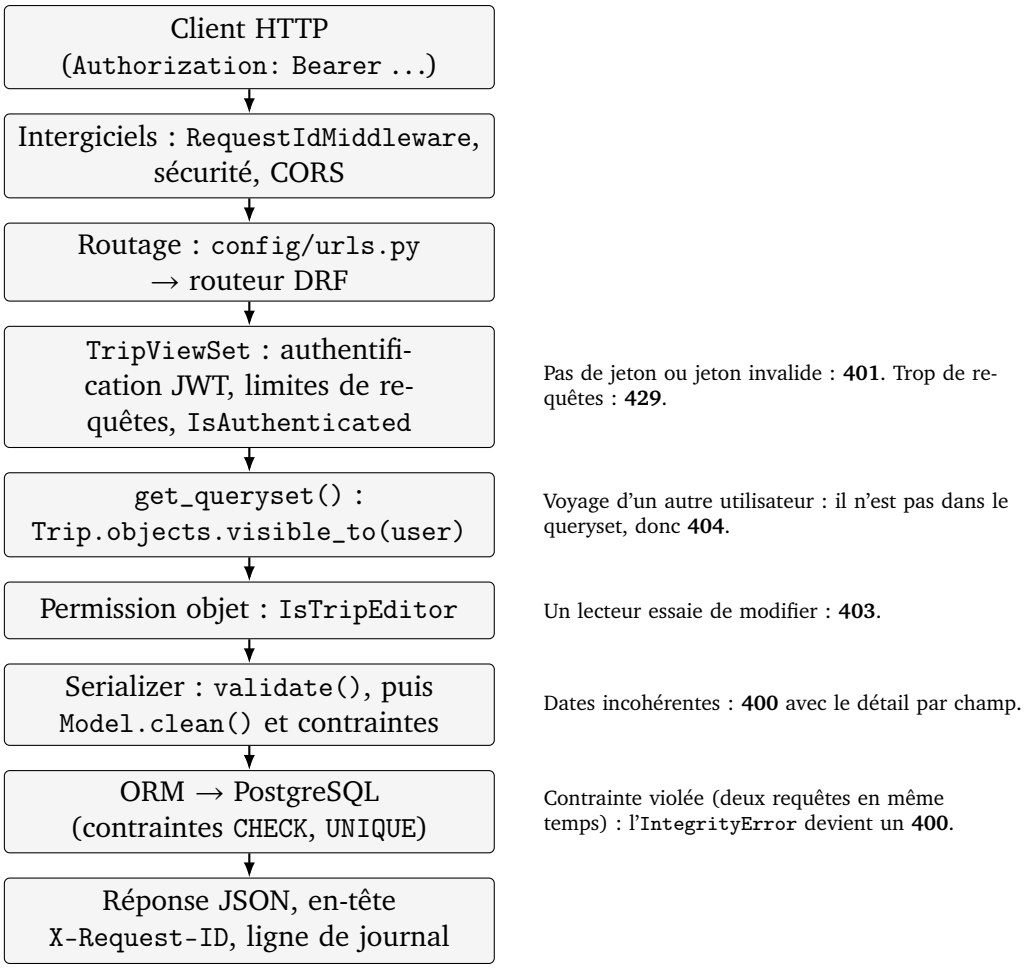


FIGURE 1 – Traitement d'une modification de voyage. Si une exception est levée, `exception_handler` la transforme en enveloppe `{"error": ...}`.

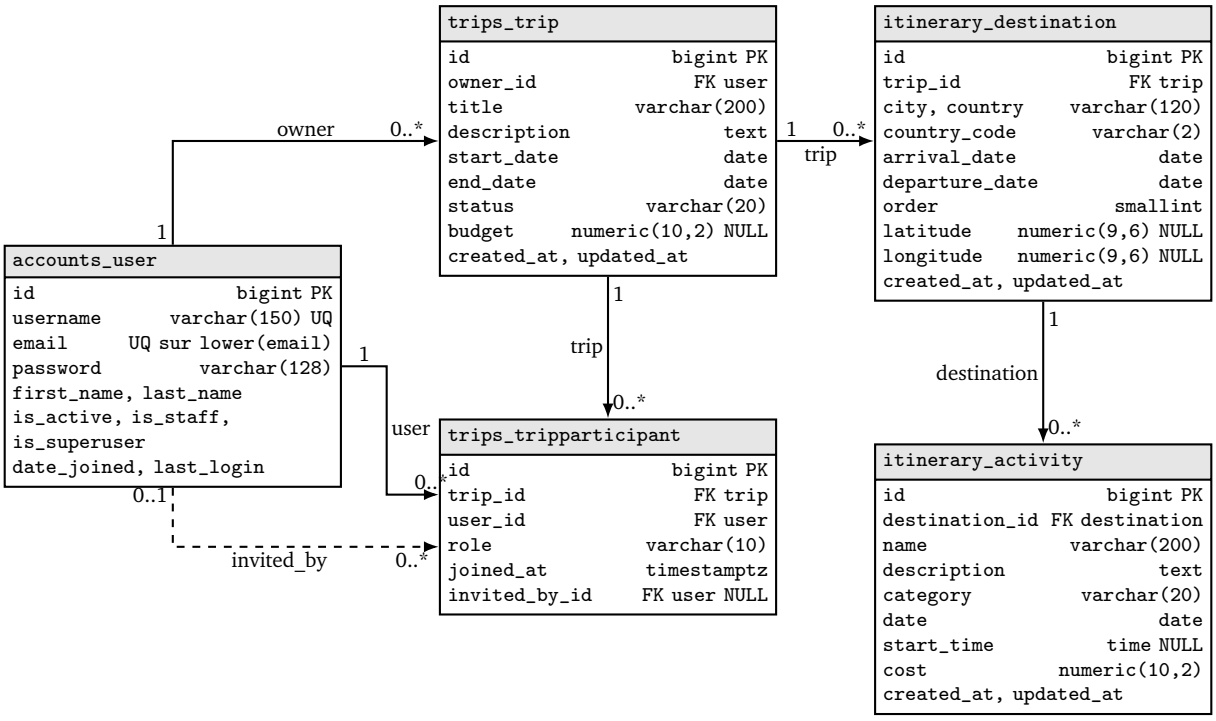


FIGURE 2 – Modèle de données de TravelHub. Chaque flèche part de la table référencée et va vers la table qui a la clé étrangère. La relation N-N entre utilisateurs et voyages passe par `trips_tripparticipant`.

`created_at` et `updated_at` viennent du modèle abstrait `TimeStampedModel` de `apps.core`.

Nous avons aussi généré le diagramme de la vraie base avec l'outil ERD de pgAdmin (figure 3). Il contient toutes les tables, y compris celles de Django et de SimpleJWT. Les cinq tables métier sont dans la partie gauche. L'image est en pleine résolution dans le PDF, on peut donc zoomer pour lire les colonnes.

4.2 Relations

Le tableau 4 donne le détail de chaque relation. Nous avons choisi des noms inverses (`related_name`) qui se lisent bien : `user.owned_trips`, `trip.destinations`, `destination.activities`.

La relation N-N demande un peu plus d'explications. Une simple table de jointure ne suffisait pas, parce que l'association a ses propres attributs (le rôle, la date d'ajout et la personne qui a invité). Nous l'avons donc modélisée avec un modèle intermédiaire, déclaré avec `through`. `TripParticipant` a deux clés étrangères vers `User`, donc il faut préciser avec `through_fields` lesquelles relient le voyage à ses participants.

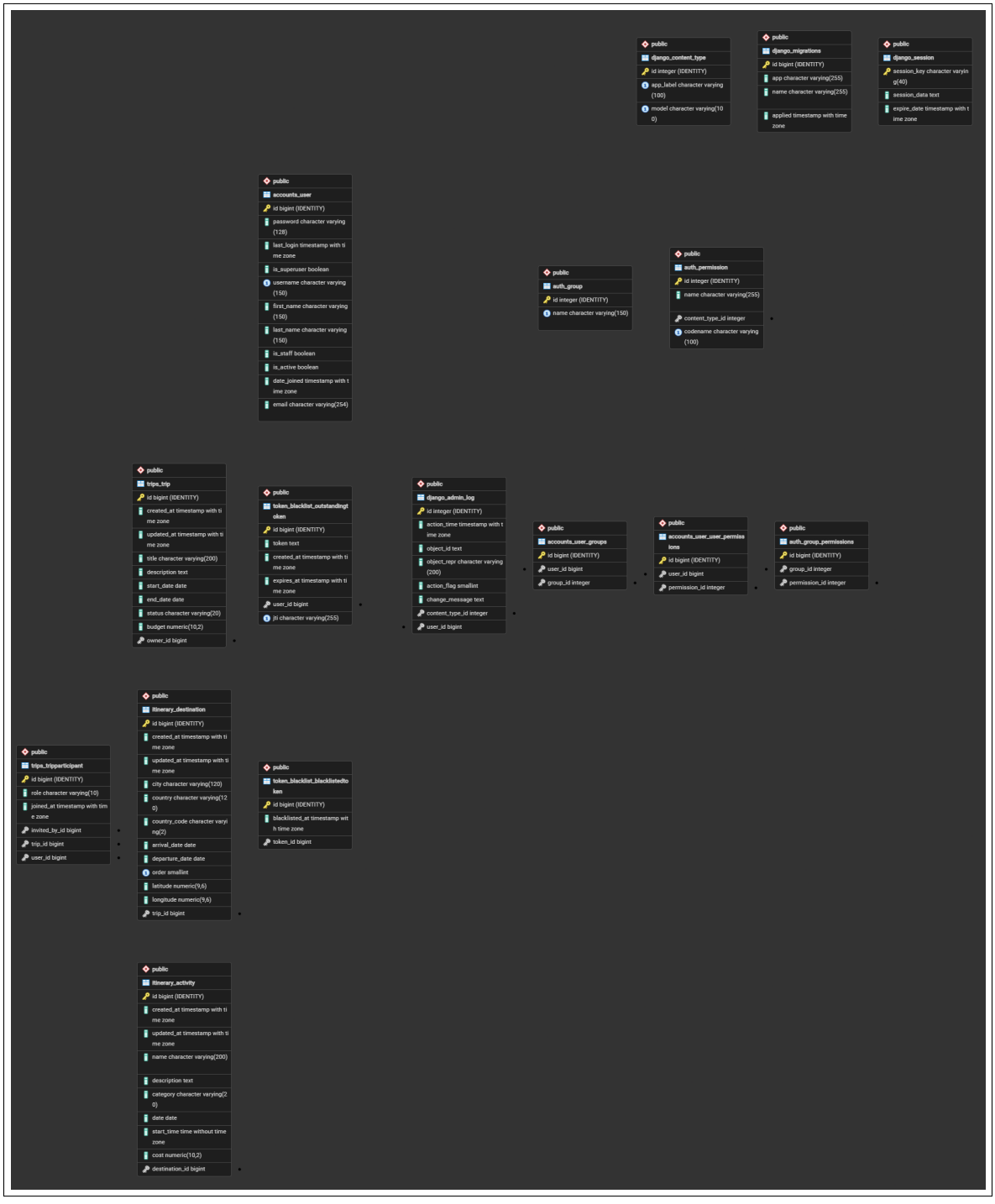


FIGURE 3 – La base `travelhub` dans l'outil ERD de pgAdmin 4 (*Tools* → *ERD Tool*). On y voit aussi les tables techniques de Django et de SimpleJWT.

TABLE 4 – Relations du modèle.

Relation	Type	Suppression	Accès inverse
`Trip.owner` → `User`	1-N	`CASCADE`	`user.owned_trips`
`Trip.participants` ↔ `User`	N-N	(aucune)	`user.trips`, via `TripParticipant`
`TripParticipant.trip` → `Trip`	1-N	`CASCADE`	`trip.memberships`
`TripParticipant.user` → `User`	1-N	`CASCADE`	`user.trip_memberships`
`TripParticipant.invited_by` → `User`	1-N	`SET_NULL`	`user.sent_trip_invitations`
`Destination.trip` → `Trip`	1-N	`CASCADE`	`trip.destinations`
`Activity.destination` → `Destination`	1-N	`CASCADE`	`destination.activities`

Extrait 2 – Relation N-N avec modèle intermédiaire (`apps/trips/models.py`, extrait).

```
participants = models.ManyToManyField(
    settings.AUTH_USER_MODEL,
    through="TripParticipant",
    # TripParticipant a deux FK vers User (user et invited_by). Il faut donc préciser
    # celles qui relient le voyage à ses participants.
    through_fields=("trip", "user"),
    related_name="trips",
)
```

La suppression en cascade suit la logique de composition. Un voyage contient ses destinations, et chaque destination contient ses activités. Par contre, si la personne qui a envoyé une invitation est supprimée, le participant invité doit rester. Dans ce cas, `invited_by` passe simplement à `NULL`.

4.3 Utilisateur personnalisé

Le modèle `accounts.User` hérite d'`AbstractUser`. Nous l'avons déclaré dans `AUTH_USER_MODEL` avant la première migration, comme Django le conseille, parce que changer de modèle utilisateur plus tard est très compliqué. Le courriel est obligatoire. Il est aussi unique sans tenir compte des majuscules, grâce à une contrainte d'unicité sur `Lower("email")`. Nous le mettons en minuscules dans `save()`, car `createsuperuser` n'appelle pas `clean()`.

4.4 Contraintes d'intégrité

Les règles importantes sont vérifiées par PostgreSQL lui-même (tableau 5). Même si une écriture passe à côté de la validation de l'application, par exemple avec deux requêtes en même temps, la base reste cohérente. Chaque contrainte a un nom et un message d'erreur en français. Le gestionnaire d'exceptions réutilise ce message (section 9).

TABLE 5 – Contraintes déclarées dans les modèles.

Nom	Type	Règle
`accounts_user_email_ci_unique`	`UNIQUE`	`lower(email)` unique
`trips_trip_end_after_start`	`CHECK`	`end_date >= start_date`
`trips_trip_budget_non_negative`	`CHECK`	budget nul ou positif
`trips_trip_status_valid`	`CHECK`	statut parmi les cinq valeurs
`trips_participant_unique_trip_user`	`UNIQUE`	un utilisateur au plus une fois par voyage
`trips_participant_single_owner`	`UNIQUE` partiel	un seul `owner` par voyage (`WHERE role = 'owner'`)
`trips_participant_role_valid`	`CHECK`	rôle parmi `owner`, `editor`, `viewer`
`itinerary_destination_unique_trip_order`	`UNIQUE`	une position par destination dans un voyage
`itinerary_destination_departure_after_arrival`	`CHECK`	`departure_date >= arrival_date`
`itinerary_destination_country_code_iso2`	`CHECK`	code pays conforme à `^[A-Z]{2}$`
`itinerary_destination_coordinates_valid`	`CHECK`	latitude et longitude données ensemble, dans leurs bornes
`itinerary_activity_cost_non_negative`	`CHECK`	`cost >= 0`
`itinerary_activity_category_valid`	`CHECK`	catégorie parmi les huit valeurs

Deux contraintes valent la peine d'être expliquées. `trips_participant_single_owner` est un index unique *partiel*, qui ne s'applique qu'aux lignes où `role = 'owner'`. La contrainte sur les coordonnées teste aussi `IS NOT NULL` de façon explicite. En SQL, une condition `CHECK` qui donne `NULL` compte comme respectée. Sans ce test, on pourrait enregistrer une latitude sans longitude.

Extrait 3 – Contraintes du voyage (`apps/trips/models.py`, extrait).

```
class Meta:
    ordering = ["start_date", "pk"]
    indexes = [models.Index(fields=["start_date"], name="trips_trip_start_date_idx")]
    constraints = [
        models.CheckConstraint(
            condition=Q(end_date__gte=F("start_date")),
            name="trips_trip_end_after_start",
            violation_error_message=(
                "La date de fin doit être postérieure ou égale à la date de début."
            ),
        ),
        models.CheckConstraint(
            condition=Q(budget__isnull=True) | Q(budget__gte=0),
            name="trips_trip_budget_non_negative",
            violation_error_message="Le budget ne peut pas être négatif.",
        ),
        # ... trips_trip_status_valid
    ]
```

Pour les montants, nous utilisons `DecimalField` (`numeric` en SQL) et jamais `float`, pour qu'un coût de 0,10 $ reste exact. Les choix (statut, rôle, catégorie) sont des `TextChoices`. Nous avons quand même ajouté une contrainte `CHECK` pour chacun, parce que Django vérifie les choix seulement à la validation et pas dans la base.

4.5 Migrations

Chaque application a une migration initiale (`0001_initial.py`) générée par Django 5.2.18 avec `makemigrations`. Avec `sqlmigrate`, on peut relire le SQL produit, par exemple pour vérifier les contraintes et l'index partiel. La commande `migrate -check` sert à vérifier qu'une base restaurée est à jour (section 12). Dans Docker, le service `web` applique les migrations à chaque démarrage, avant de lancer Gunicorn.

5 Gestion des voyages, destinations, activités et participants

5.1 QuerySets réutilisables

Nous avons regroupé les requêtes qui reviennent souvent dans un `QuerySet` personnalisé, accessible par `Trip.objects` (`TripQuerySet.as_manager()`). On peut les enchaîner.

TABLE 6 – Méthodes de `TripQuerySet`.

Méthode	Résultat
`visible_to(user)`	Voyages dont l'utilisateur est propriétaire ou participant, quel que soit son rôle (rien pour un anonyme)
`editable_by(user)`	Voyages que l'utilisateur peut modifier (propriétaire, rôle `owner` ou `editor`)
`upcoming()`	Voyages à venir, triés par date, sans les voyages annulés ou terminés
`with_totals()`	Ajoute `destination_count`, `activity_count` et `total_cost`

Toutes les vues et tous les resolvers GraphQL commencent par `visible_to()`. Cette méthode utilise une sous-requête `Exists()` au lieu d'une jointure sur les participants. Avec une jointure, chaque voyage apparaîtrait une fois par participant, et les sommes de `with_totals()` seraient fausses.

Extrait 4 – Voyages visibles et totaux (`apps/trips/models.py`, extrait).

```python
def visible_to(self, user):
    if user is None or not user.is_authenticated:
        return self.none()
    # Exists() au lieu d'une jointure, pour éviter les doublons et garder with_totals() exact.
    membership = TripParticipant.objects.filter(trip=OuterRef("pk"), user=user)
    return self.filter(Q(owner=user) | Exists(membership))

def with_totals(self):
    return self.annotate(
        destination_count=Count("destinations", distinct=True),
        activity_count=Count("destinations__activities", distinct=True),
        total_cost=Coalesce(
            Sum("destinations__activities__cost"),
            Value(0),
            output_field=DecimalField(max_digits=12, decimal_places=2),
        ),
    )
```

`total_cost` se calcule en suivant le chemin voyage → destinations → activités. Dans cette jointure, chaque activité n'apparaît qu'une fois, donc la somme est juste. Il ne faut pas ajouter un `Count()` sur une autre relation, parce que la jointure en plus multiplierait les lignes. C'est aussi pour ça que la liste des voyages ne permet pas de trier sur un champ de relation.

5.2 Relations inverses et préchargement

Grâce aux relations inverses du tableau 4, on n'a pas besoin d'écrire de requêtes à la main. Par exemple, `trip.memberships.select_related("user")` donne les participants et `destination.activities.all()` donne les activités d'une étape. On peut aussi filtrer à travers les relations, dans les deux sens, comme avec `Activity.objects.filter(destination__trip=trip)`.

Le préchargement évite le problème des « N+1 requêtes ». La liste des voyages charge

le propriétaire avec `select_related("owner")`. Le détail d'un voyage précharge ses destinations, triées par position, puis leurs activités, avec un objet `Prefetch`. Le nombre de requêtes SQL reste donc le même, peu importe le nombre de destinations.

5.3 Agrégations

L'action `GET /api/trips/{id}/stats/` calcule le budget restant et les coûts par catégorie. Nous laissons PostgreSQL faire tous les calculs, avec `aggregate()` pour les totaux et `values().annotate()` pour le regroupement.

Extrait 5 – Statistiques par catégorie (`apps/trips/services.py`, extrait).

```
activities = Activity.objects.filter(destination__trip=trip)
totals = activities.aggregate(count=Count("pk"), total=Sum("cost"), average=Avg("cost"))
by_category = (
    activities.values("category")
    .annotate(count=Count("pk"), total_cost=Sum("cost"), average_cost=Avg("cost"))
    # Tri explicite, sinon l'ordre par défaut d'Activity ajouterait date au GROUP BY.
    .order_by("-total_cost", "category")
)
```

Le commentaire de l'extrait 5 parle d'un piège de l'ORM dans lequel nous sommes tombés. Par défaut, les activités sont triées par date. Si on garde ce tri, Django ajoute la date au `GROUP BY` et chaque catégorie est découpée jour par jour.

5.4 Services métier

Création. Le service `create_trip()` valide le voyage avec `full_clean()` et l'enregistre. Il ajoute ensuite le propriétaire comme participant `owner`, dans la même transaction. Un signal `post_save` aurait pu faire la même chose, mais de façon cachée. Avec un service, on voit l'effet de bord directement à l'appel.

Clonage. Le service `clone_trip()` copie un voyage, ses destinations et leurs activités dans un nouveau brouillon. Il utilise `bulk_create()`, qui insère chaque niveau en une seule requête. Avec PostgreSQL, `bulk_create()` renvoie les clés primaires créées. Les destinations copiées peuvent donc servir tout de suite de parents aux activités copiées.

Extrait 6 – Clonage dans une transaction (`apps/trips/services.py`, extrait).

```python
@transaction.atomic
def clone_trip(source: Trip, *, owner, title=None) -> Trip:
    trip = create_trip(owner=owner, title=title or f"Copie de {source.title}", ...,
                       status=Trip.Status.DRAFT)
    destinations = list(source.destinations.prefetch_related("activities"))
    # Avec PostgreSQL, bulk_create renvoie les clés primaires. On peut donc utiliser
    # les copies comme parents.
    copies = Destination.objects.bulk_create(_copy(d, trip=trip) for d in destinations)
    Activity.objects.bulk_create(
        _copy(activity, destination=copy)
        for original, copy in zip(destinations, copies, strict=True)
        for activity in original.activities.all()
    )
    return trip
```

Participants. Le propriétaire ajoute un participant avec son courriel et lui donne le rôle `editor` ou `viewer`. On ne peut pas donner le rôle `owner` au moment de l'invitation. Pour transférer la propriété, on change le rôle ensuite, et l'ancien propriétaire devient éditeur. On ne peut pas non plus retirer ou rétrograder directement le propriétaire, sinon le voyage n'en aurait plus. Si on ajoute deux fois la même personne, la contrainte `trips_participant_unique_trip_user` est violée. Nous laissons la base décider, comme ça deux ajouts en même temps ne posent pas de problème.

5.5 Administration et données de démonstration

L'administration de Django (`/admin/`) permet de gérer tous les modèles, regroupés par application (figure 4). Dans la fiche d'un voyage, les participants et les destinations sont affichés en ligne. La commande `seed_demo` crée trois comptes et quatre voyages (Montréal, Paris, Tokyo, Dakar) qui couvrent tous les rôles. On peut la relancer sans créer de doublons. Ses dates dépendent du jour où on la lance, ce qui donne un voyage terminé, un en cours, un planifié et un brouillon.

6 Intégrations externes

TravelHub utilise deux API publiques qui ne demandent pas de clé. Open-Meteo [12, 11] donne le géocodage et la météo avec une interface REST. Countries [1] décrit les pays avec une interface GraphQL. Tout le code d'intégration est dans `apps/integrations/services/`. Les clients renvoient des `dataclasses` immuables et jamais le JSON brut du fournisseur.

6.1 Open-Meteo (REST)

Pour avoir la météo d'une destination, il faut deux appels.

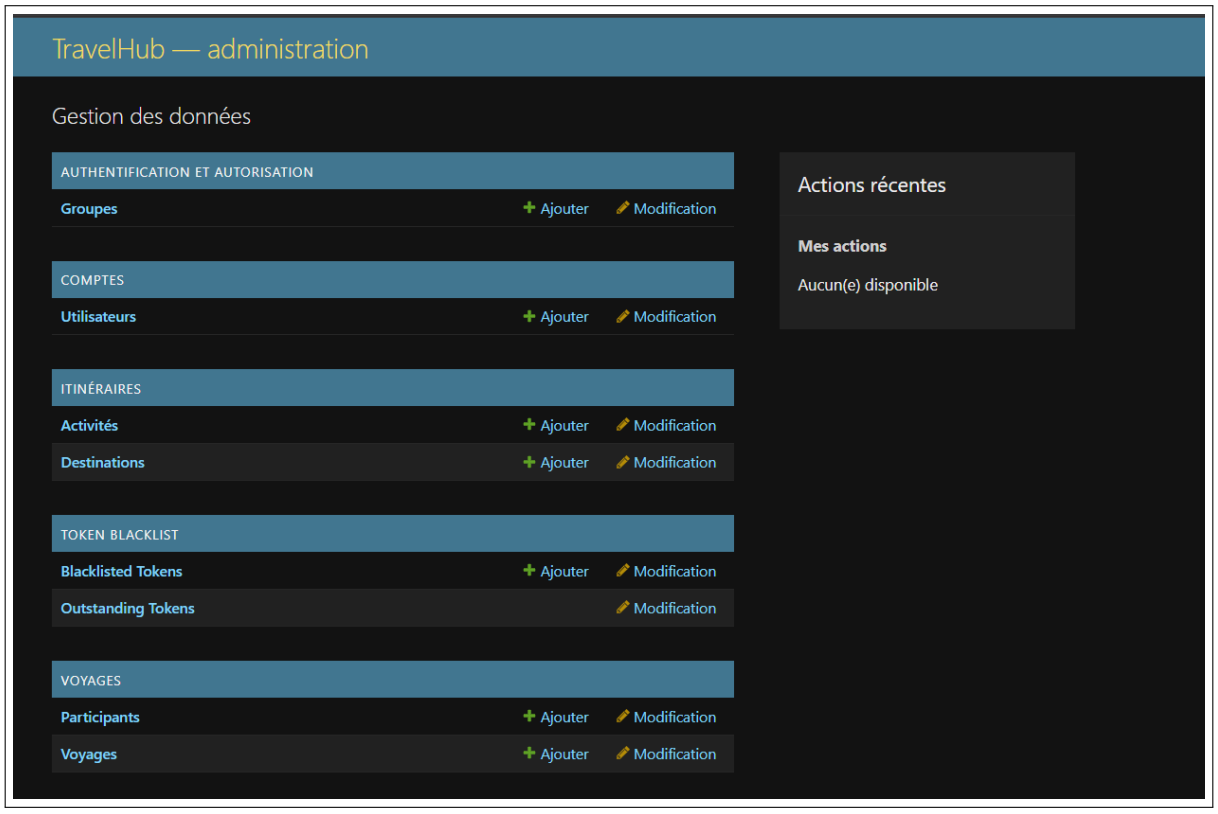


FIGURE 4 – Page d'accueil de l'administration Django. Les modèles sont regroupés par application (comptes, itinéraires, voyages, liste noire des jetons).

Le premier est le géocodage. Si une destination a été saisie sans coordonnées, on la cherche d'abord avec `geocoding-api.open-meteo.com/v1/search`, à partir de la ville et du code pays. Les coordonnées trouvées sont enregistrées dans la destination, ce qui évite de refaire cet appel ensuite.
Le deuxième appel, `api.open-meteo.com/v1/forecast`, renvoie les conditions actuelles et les prévisions des 16 prochains jours. Le service garde seulement les jours du séjour.
Les codes météo WMO sont traduits en français (« Pluie modérée », « Orage »…). Si le séjour est complètement en dehors des 16 jours, on renvoie l'erreur `no_data_available`. Nous gardons un jour de marge, car la date du serveur et la date locale de la destination ne sont pas toujours les mêmes.
Pour enregistrer les coordonnées, nous utilisons `update()` avec un filtre sur `latitude IS NULL`, et pas `save()`. Comme ça, on écrit seulement les deux colonnes concernées et on n'écrase pas une autre modification de la destination faite au même moment.

6.2 Countries (GraphQL)

Le client Countries envoie une requête GraphQL en `POST` à `countries.trevorblades.com/graphql`. Le code pays est passé dans `variables` et n'est jamais collé dans le texte de la requête. C'est la même idée qu'une requête SQL paramétrée, et cela empêche les injections.

Extrait 7 – Requête GraphQL paramétrée (`apps/integrations/services/countries.py`, extrait raccourci).

```
COUNTRY_QUERY = """
query Country($code: ID!) {
  country(code: $code) {
    code name native capital emoji currency
    continent { name }
    languages { name native }
  }
}
"""
body = {"query": COUNTRY_QUERY, "variables": {"code": code}}
```

En général, un serveur GraphQL répond 200 même quand il y a une erreur. Le client regarde donc la liste `errors` avant le champ `data`. Si cette liste n'est pas vide, on renvoie une erreur 502. Pour un code inconnu, le service répond `{"country": null}`, que nous traduisons en `country_not_found` (404).

6.3 Couche HTTP commune

Les deux clients utilisent le même module, `apps/integrations/services/http.py`. Chaque requête a un délai de 5 secondes et peut être essayée deux fois. Le deuxième essai a lieu seulement si l'erreur est passagère.

TABLE 7 – Nouvel essai et traduction des échecs.

Situation	Nouvel essai	Exception	HTTP
Délai dépassé	oui	`ExternalServiceTimeout`	504
Erreur réseau	oui	`ExternalServiceUnavailable`	503
Réponse 5xx	oui	`ExternalServiceUnavailable`	503
Réponse 4xx	non	`ExternalServiceUnavailable`	503
JSON illisible ou de forme inattendue	non	`ExternalServiceBadResponse`	502
Ville ou pays inconnu	non	`LocationNotFound`, `CountryNotFound`	404
Séjour hors des 16 jours	non	`NoDataAvailable`	404

On ne réessaie pas après une réponse 4xx, car la même requête serait refusée de la même façon. Les exceptions de `apps/integrations/exceptions.py` ne dépendent pas de DRF. Elles contiennent un statut HTTP et un code stable, et chaque API (REST ou GraphQL) les traduit à sa manière. Chaque essai est journalisé avec le service, l'URL sans paramètres, le statut, la durée et le numéro de l'essai.

6.4 Cache

Les réponses sont gardées dans le cache de Django, mais seulement si l'analyseur les a acceptées. Une erreur passagère ne reste donc pas bloquée dans le cache jusqu'à l'expiration de l'entrée.

TABLE 8 – Durées de mise en cache.

Donnée	Durée	Clé
Géocodage	7 jours	code pays et condensat SHA-256 du nom de ville
Prévisions	30 minutes	position arrondie à 4 décimales (environ 10 m)
Pays	24 heures	code pays

Nous hachons le nom de la ville pour que la clé soit valide avec n'importe quel moteur de cache, même si l'utilisateur tape des caractères spéciaux. En arrondissant la position, deux destinations très proches partagent la même entrée.

6.5 Agrégation et dégradation partielle

L'action `GET /api/destinations/{id}/overview/` regroupe dans une seule réponse la destination, son voyage, ses activités, le coût total, la météo et le pays. Les deux services externes sont appelés en parallèle avec un `ThreadPoolExecutor`, pendant que le fil principal lit la base.

Extrait 8 – Appels en parallèle (`apps/integrations/services/aggregator.py`, extrait).

```
with ThreadPoolExecutor(max_workers=len(SOURCES), thread_name_prefix="overview") as pool:
    futures = {
        WEATHER: pool.submit(contextvars.copy_context().run, fetch_forecast, destination),
        COUNTRY: pool.submit(
            contextvars.copy_context().run, countries.get_country, destination.country_code
        ),
    }
    activities = list(destination.activities.all())  # pendant les appels externes
    results, warnings = {}, []
    for source, future in futures.items():
        try:
            results[source] = future.result()
        except IntegrationError as exc:
            results[source] = None
            warnings.append(SourceWarning(source, exc.code, exc.message))
```

Ce parallélisme nous a obligés à faire attention à quelques points. D'abord, les fils secondaires ne font aucune requête SQL. Sinon, chaque fil ouvrirait sa propre connexion, qui ne serait jamais fermée, et il ne verrait pas la transaction du fil principal. Le géocodage calcule donc les coordonnées sans les écrire, et c'est le fil principal qui les enregistre après.

Ensuite, chaque tâche tourne dans une copie du contexte (`contextvars`). Ses lignes de journal gardent ainsi l'identifiant de la requête en cours.

Enfin, si une source échoue, la réponse ne plante pas et reste un 200. Le bloc de cette source vaut `null` et la liste `warnings` explique ce qui s'est passé (extrait 9).

Extrait 9 – Dégradation partielle quand Open-Meteo est indisponible.

```
"weather": null,
"warnings": [
  {"source": "weather", "code": "external_service_unavailable",
   "message": "Open-Meteo est momentanément indisponible (erreur 503)."}
]
```

Cette dégradation partielle sera utile pour le front-end. Même si un fournisseur est en panne, la page d'une destination pourra quand même s'afficher.

7 API REST

7.1 Points d'accès

Nous avons construit l'API REST avec Django REST Framework [2]. Les tableaux 9 à 11 listent ses points d'accès. Ils demandent tous un jeton d'accès (`Authorization: Bearer <access>`), sauf l'inscription, la connexion, le rafraîchissement et la documentation.

TABLE 9 – Authentification (`apps/accounts/urls.py`).

Méthode	Chemin	Rôle
POST	`/api/auth/register/`	Inscription, renvoie l'utilisateur et une paire de jetons
POST	`/api/auth/token/`	Connexion, donne un jeton d'accès (15 min) et un jeton de rafraîchissement (1 jour)
POST	`/api/auth/token/refresh/`	Nouveau jeton d'accès, l'ancien jeton de rafraîchissement est révoqué
POST	`/api/auth/logout/`	Révoque son propre jeton de rafraîchissement
GET	`/api/auth/me/`	Profil de l'utilisateur connecté

TABLE 10 – Voyages et participants (`apps/trips/urls.py`).

Méthode	Chemin	Rôle
GET, POST	`/api/trips/`	Mes voyages (20 par page) et création
GET, PUT, PATCH, DELETE	`/api/trips/{id}/`	Détail avec destinations et activités, modification, suppression
GET	`/api/trips/{id}/stats/`	Coût total, budget restant, coûts par catégorie
POST	`/api/trips/{id}/clone/`	Copie du voyage en brouillon, dans une transaction
GET, POST	`/api/trips/{id}/participants/`	Liste et ajout par courriel
PATCH, DELETE	`/api/trips/{id}/participants/{user_id}/`	Changement de rôle et retrait

TABLE 11 – Destinations et activités (`apps/itinerary/urls.py`).

Méthode	Chemin	Rôle
GET, POST	`/api/destinations/`	Destinations de mes voyages et création
GET, PUT, PATCH, DELETE	`/api/destinations/{id}/`	Détail, modification, suppression
GET	`/api/destinations/{id}/weather/`	Météo du séjour (Open-Meteo)
GET	`/api/destinations/{id}/country/`	Fiche du pays (Countries)
GET	`/api/destinations/{id}/overview/`	Vue d'ensemble avec dégradation partielle
GET, POST	`/api/activities/`	Activités de mes voyages et création
GET, PUT, PATCH, DELETE	`/api/activities/{id}/`	Détail, modification, suppression

Il y a aussi `/api/schema/` (schéma OpenAPI), `/api/docs/` (Swagger UI), `/api/redoc/` (ReDoc), `/graphql/` et `/health/`.

7.2 Serializers

Les serializers transforment les modèles en JSON et valident ce que le client envoie. Ils n'acceptent que les champs d'une liste blanche. Les champs calculés ou sensibles, comme `owner`, `id`, les horodatages et les totaux, sont en lecture seule. Un client ne peut donc pas se donner un voyage en envoyant `"owner": 7`.

Pour le voyage, nous avons trois serializers. `TripSerializer` sert pour la liste, la création et la modification. `TripDetailSerializer` ajoute les destinations et leurs activités, imbriquées en lecture seule. Enfin, des serializers sans modèle décrivent les réponses calculées, comme `TripStatsSerializer`. Le mixin `ModelCleanMixin`, expliqué à la section 9, applique en plus `Model.clean()` et les contraintes du modèle.

7.3 ViewSets et routage

Chaque ressource est un `ModelViewSet` enregistré dans un `DefaultRouter`, qui crée les routes de liste et de détail. Les opérations qui ne sont pas du CRUD sont des actions déclarées avec `@action` : `stats`, `clone`, `participants`, `weather`, `country` et `overview`.

Extrait 10 – Queryset et permissions du ViewSet des voyages (`apps/trips/views.py`, extrait).

```python
class TripViewSet(TripRolePermissionMixin, viewsets.ModelViewSet):
    filterset_class = TripFilter
    search_fields = ["title", "description"]
    ordering_fields = ["start_date", "title", "created_at"]
    object_permission_by_action = {
        "update": IsTripEditor,
        "partial_update": IsTripEditor,
        "destroy": IsTripOwner,
        "participant_detail": IsTripOwner,
    }

    def get_queryset(self):
        queryset = Trip.objects.visible_to(self.request.user).with_totals().select_related("owner")
        if self.action == "retrieve":
            queryset = queryset.prefetch_related(
                Prefetch("destinations",
                         queryset=Destination.objects.order_by("order").prefetch_related("activities"))
            )
        return queryset
```

Le dictionnaire `object_permission_by_action` associe chaque action à sa permission objet. Si une action n'y est pas (en pratique, une lecture), il suffit d'être membre du voyage. On voit donc toutes les règles d'accès au même endroit.

7.4 Filtres, recherche, tri et pagination

Trois mécanismes de filtrage sont actifs par défaut. Il y a `django-filter`, la recherche texte de DRF et un tri stable qui départage toujours par identifiant. Les listes sont paginées par 20.

TABLE 12 – Paramètres des listes.

Ressource	Filtres	Tri (`ordering`)
Voyages	`status`, `start_date__gte`, `end_date__lte`, `title__icontains`, `search`	date, titre, création
Destinations	`trip`, `country_code`, `arrival_date__gte`, `departure_date__lte`, `search`	ordre, arrivée, ville
Activités	`trip`, `destination`, `category`, `date`, `date__gte`, `date__lte`, `search`	coût, date, nom

Les filtres `?trip=` et `?destination=` sont de simples filtres sur un nombre, appliqués à un queryset déjà limité aux voyages de l'utilisateur. Avec l'identifiant du voyage de quelqu'un d'autre, on obtient une liste vide et pas une erreur. Le filtre ne permet donc pas de savoir si ce voyage existe.

7.5 Codes HTTP et format des erreurs

L'API utilise les codes HTTP pour ce qu'ils veulent dire (tableau 13).

TABLE 13 – Codes HTTP renvoyés.

Code	Code d'erreur	Situation
200, 201, 204	(aucun)	Lecture ou modification, création, suppression
400	`validation_error`, `integrity_error`	Données invalides, contrainte violée
401	`not_authenticated`, `token_not_valid`	Jeton absent, invalide ou expiré
403	`permission_denied`	Membre du voyage qui n'a pas le bon rôle
404	`not_found`	Ressource absente ou qui appartient à quelqu'un d'autre
405, 415	`method_not_allowed`, `unsupported_media_type`	Méthode ou type de contenu refusé
429	`throttled`	Limite de requêtes atteinte
502, 503, 504	`external_service_*`	Service externe illisible, indisponible ou trop lent

Toutes les erreurs ont la même enveloppe. Le champ `code` est un identifiant stable que le programme client peut tester. Le champ `message` est destiné à une personne, et `details` donne les erreurs champ par champ.

Extrait 11 – Enveloppe d'erreur d'une validation.

```
{
  "error": {
    "code": "validation_error",
    "message": "Les données envoyées sont invalides.",
    "details": {"end_date": ["La date de fin doit être postérieure ou égale à la date de début."]}
  }
}
```

C'est le gestionnaire `apps.core.exceptions.exception_handler` qui produit cette forme. Il convertit aussi deux exceptions qui, sans lui, donneraient une erreur 500 : la `ValidationError` de Django levée par les services et l'`IntegrityError` levée par PostgreSQL. Les URL inconnues et les erreurs inattendues en dehors de DRF (`handler404`, `handler500`) renvoient la même enveloppe, sans page HTML ni trace de pile.

7.6 Documentation OpenAPI

drf-spectacular [5] génère un schéma OpenAPI 3 à partir des ViewSets et des serializers. Nous avons annoté chaque opération avec `@extend_schema` pour ajouter un résumé en français, les réponses d'erreur possibles et des exemples. L'option `COMPONENT_SPLIT_REQUEST` sépare les schémas de lecture et d'écriture, pour que les champs calculés n'apparaissent pas dans les corps de requête. Dans Swagger UI (figure 5), on peut tester l'API après avoir collé un jeton avec le bouton *Authorize*.



FIGURE 5 – Swagger UI (`/api/docs/`). En haut, la description de l'API et le bouton *Authorize*. En dessous, les opérations sur les activités.

8 API GraphQL

8.1 Schéma

Nous avons construit l'API GraphQL avec Graphene-Django [7], en suivant la spécification GraphQL [8]. Elle n'a qu'un seul point d'entrée, `POST /graphql/`. Chaque application déclare ses types, ses requêtes et ses mutations dans son propre `schema.py`, et le schéma racine les assemble par héritage multiple.

Extrait 12 – Assemblage du schéma (`config/schema.py`).

```
class Query(itinerary.Query, trips.Query, accounts.Query, graphene.ObjectType):
    pass


class Mutation(itinerary.Mutation, trips.Mutation, graphene.ObjectType):
    pass


schema = graphene.Schema(query=Query, mutation=Mutation)
```

Les types `User`, `Trip`, `TripParticipant`, `Destination` et `Activity` héritent de `DjangoObjectType`. Chacun a sa liste blanche de champs. Par exemple, le type `User` ne montre ni le mot de passe ni les droits d'administration, et le courriel n'est visible que par son titulaire. Les choix des modèles deviennent des énumérations GraphQL (`TripStatus`, `ParticipantRole`, `ActivityCategory`). Les données externes ont leurs propres types, `Weather`, `CountryInfo` et `DestinationOverview`.

TABLE 14 – Requêtes et mutations.

Racine	Champs
Query	`me`, `myTrips(status, startAfter, search, orderBy, limit, offset)`, `myTripsCount`, `trip(id)`, `destination(id)`, `destinationOverview(id)`
Mutation	`createTrip`, `updateTrip`, `deleteTrip`, `cloneTrip`, `addParticipant`, `removeParticipant`, `createDestination`, `updateDestination`, `deleteDestination`, `createActivity`, `updateActivity`, `deleteActivity`

8.2 Réutilisation de la logique REST

Nous n'avons réécrit aucune règle pour GraphQL. Tous les resolvers partent de `Trip.objects.visible_to(user)`. Les mutations réutilisent les serializers DRF (et donc `Model.clean()` et les contraintes) ainsi que les services (`create_trip`, `clone_trip`, `add_participant`). Les permissions objet sont les mêmes classes qu'en REST. Chaque mutation tourne dans une transaction (`@transaction.atomic`).

Extrait 13 – Mutation de modification (`apps/trips/schema.py`, extrait).

```python
class UpdateTrip(graphene.Mutation):
    class Arguments:
        id = graphene.ID(required=True)
        input = TripUpdateInput(required=True)

    trip = graphene.Field(TripType)

    @classmethod
    @transaction.atomic
    def mutate(cls, root, info, id, input):
        trip = get_or_not_found(visible_trips(info), TRIP_NOT_FOUND, pk=id)
        check_object_permission(info, IsTripEditor, trip)
        serializer = TripSerializer(trip, data=dict(input), partial=True,
                                    context=serializer_context(info))
        return cls(trip=reload_trip(info, save_serializer(serializer)))
```

8.3 Authentification, limites et erreurs

La vue `TravelHubGraphQLView` hérite de `GraphQLView`. Comme pour l'API REST, elle vérifie le jeton avec la classe `JWTAuthentication` de SimpleJWT. Sans en-tête, la requête est anonyme et chaque champ protégé répond `not_authenticated`. Un jeton invalide ou expiré donne un 401. La vue ne tient jamais compte du cookie de session, c'est pour ça qu'on peut l'exempter de la protection CSRF sans risque. Les limites de requêtes utilisent les classes et les compteurs de DRF. Un utilisateur a donc un seul quota pour REST et GraphQL.
Les erreurs gardent le code et les détails de l'erreur REST équivalente dans le champ `extensions`. Si l'erreur est inattendue, son message n'est pas montré. Elle devient `internal_error` et sa trace est journalisée.

Extrait 14 – Erreur de validation d'une mutation `updateTrip`.

```
{
  "errors": [{
    "message": "Les données envoyées sont invalides.",
    "path": ["updateTrip"],
    "extensions": {
      "code": "validation_error",
      "details": {"endDate": ["La date de fin doit être postérieure ou égale à la date de début."]}
    }
  }],
  "data": {"updateTrip": null}
}
```

8.4 Durcissement et performance

Trois paramètres, relus à chaque requête, protègent le point d'entrée. `GRAPHQL_MAX_DEPTH` limite la profondeur des requêtes à 6, ce qui bloque les requêtes cycliques (`trip { destinations { trip { ... } } }`) avant qu'elles s'exécutent. `GRAPHQL_INTROSPECTION` ferme l'introspection (`__schema`, `__type`) en production, mais laisse

passer `__typename`, que des clients comme Apollo ajoutent à chaque requête. Enfin, avec `GRAPHQL_GRAPHIQL`, l'interface GraphiQL n'est servie qu'en développement.

Pour la performance, `myTrips` et `trip` préchargent les destinations, les activités ou les participants seulement si ces champs sont demandés. La fonction `selected_fields()` parcourt la sélection, fragments compris. Le nombre de requêtes SQL ne dépend donc pas du nombre de voyages renvoyés. `myTrips` est paginée comme l'API REST, avec 20 éléments par défaut et 100 au maximum.

8.5 Exemple

La requête de l'extrait 15 récupère en un seul aller-retour les voyages planifiés, leurs destinations, la météo actuelle et la fiche de chaque pays. En REST, il faudrait une requête pour la liste, puis deux requêtes par destination.

Extrait 15 – Requête GraphQL qui regroupe plusieurs ressources.

```
query MesVoyagesPlanifies {
  myTripsCount(status: PLANNED)
  myTrips(status: PLANNED, orderBy: START_DATE, limit: 5) {
    id
    title
    startDate
    totalCost
    destinations {
      city
      weather { current { temperature description } }
      countryInfo { name emoji currencies }
    }
  }
}
```

En développement, on peut écrire ces requêtes dans GraphiQL (figure 6). L'explorateur à gauche liste les champs racine du schéma et aide à construire la requête. Le jeton se met dans l'onglet *Headers*.

8.6 Comparaison REST et GraphQL

Le tableau 15 compare les deux API telles que nous les avons faites dans TravelHub. REST, au sens de Fielding [6], organise l'API autour de ressources qui ont chacune leur adresse. GraphQL l'organise autour d'un schéma typé que le client interroge.

Pour nous, les deux API ne sont pas en concurrence, elles se complètent. REST est pratique pour les opérations simples et pour les autres applications qui voudraient se brancher, grâce au cache HTTP et à OpenAPI. GraphQL sera plus pratique pour les écrans du front-end qui affichent plusieurs ressources à la fois. Offrir les deux ne coûte pas cher, puisque toute la logique est partagée.

TABLE 15 – Comparaison des deux API de TravelHub.

Critère	**REST (DRF)**	**GraphQL (Graphene)**
Points d'entrée	Une URL par ressource et par action (18 chemins)	Un seul, `POST /graphql/`
Forme de la réponse	Décidée par le serveur (serializer)	Choisie par le client, champ par champ
Trop ou pas assez de données	La liste n'a pas le détail. Nous avons ajouté `overview` pour éviter les allers-retours	Le client demande exactement les champs qu'il veut
Erreurs	Code HTTP (400, 403, 404…) et enveloppe `error`	HTTP 200 et liste `errors` avec `extensions.code`, 400 pour une requête invalide
Échec partiel	Tout ou rien, sauf `overview` (`warnings`)	Prévu par GraphQL. Un champ en échec vaut `null` et les autres sont servis
Cache HTTP	Possible sur les `GET` (URL stables)	Difficile, car tout passe par `POST`
Documentation	OpenAPI, Swagger UI, ReDoc	Schéma typé et GraphiQL, introspection fermée en production
Pagination	Numéro de page (`?page=`)	`limit` et `offset`, avec `myTripsCount`
Risques propres	Champs exposés en trop (affectation de masse)	Requêtes coûteuses. Profondeur limitée à 6, mais largeur non limitée
Authentification, permissions	JWT, `visible_to()`, classes de `apps.core.permissions`	Les mêmes, avec un quota partagé avec REST
Validation	Serializers, `Model.clean()`, contraintes	La même, les mutations appellent les mêmes serializers

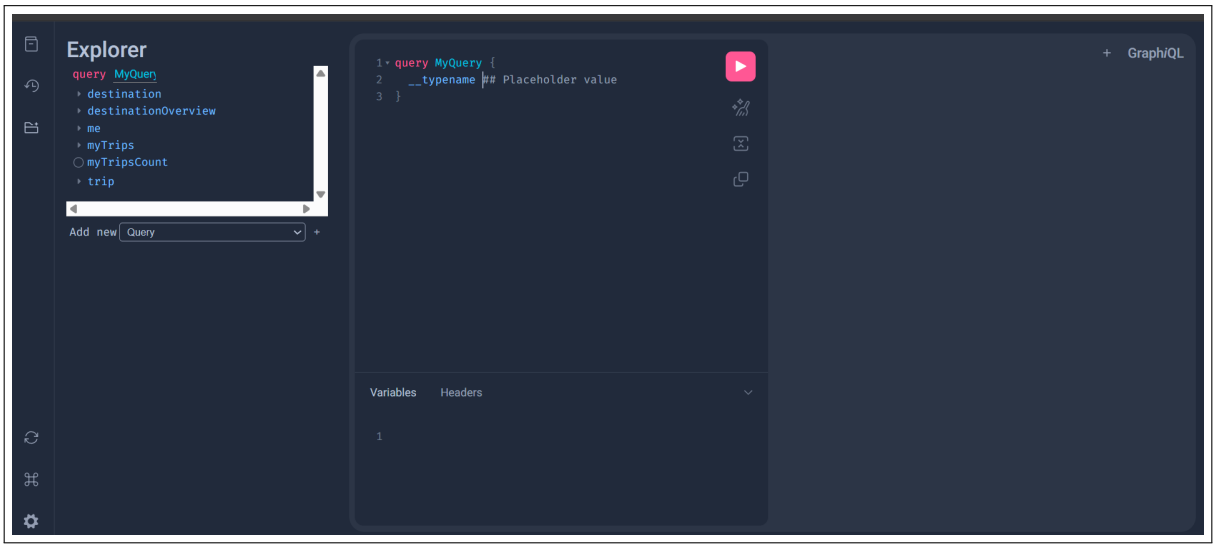


FIGURE 6 – GraphiQL sur `/graphql/` en développement. L'explorateur liste les champs racine de `Query` (`destination`, `destinationOverview`, `me`, `myTrips`, `myTripsCount`, `trip`).

9 Validation, transactions et intégrité

9.1 Trois niveaux de validation

Avant d'être écrite, une donnée passe trois contrôles l'un après l'autre (tableau 16). Chacun a son utilité, et aucun ne suffirait tout seul.

TABLE 16 – Niveaux de validation.

Niveau	Règles	Pourquoi
Serializer	Types, longueurs, choix, champs en lecture seule, cohérence des dates en `PATCH`	Refuser tôt, avec un message par champ
`Model.clean()`	Destination dans les dates du voyage, activité dans les dates de la destination, pas de raccourcissement qui laisserait un enfant en dehors	Règles métier qui ont besoin de lire d'autres lignes
Contraintes PostgreSQL	`CHECK` et `UNIQUE` du tableau 5	Dernière protection, qui marche aussi quand deux requêtes arrivent en même temps

En `PATCH`, le client peut n'envoyer qu'une des deux dates. Le serializer reprend alors l'autre date dans la valeur enregistrée avant de les comparer. Sinon, une date de fin

plus tôt que la date de début existante passerait la validation et échouerait seulement dans la base.

9.2 Appliquer `Model.clean()` depuis DRF

DRF n'appelle ni `Model.clean()` ni `validate_constraints()`. Sans rien faire, l'API ignorerait donc les règles de `clean()`, et une contrainte violée donnerait une `IntegrityError` au moment de l'écriture. Nous avons écrit le mixin `ModelCleanMixin` pour régler ce problème.

Extrait 16 – Validation par le modèle (`apps/core/serializers.py`, extrait).

```
class ModelCleanMixin:
    def validate(self, attrs):
        attrs = super().validate(attrs)
        if self.instance is not None:
            candidate = copy.copy(self.instance)
        else:
            candidate = self.Meta.model()
        for field, value in attrs.items():
            setattr(candidate, field, value)
        try:
            candidate.clean()
            candidate.validate_constraints()
        except DjangoValidationError as exc:
            raise serializers.ValidationError(django_error_detail(exc)) from exc
        return attrs
```

Les règles sont testées sur une *copie* de l'instance. Si la validation échoue, l'objet chargé par la vue ne change pas. Les messages de Django sont remis au format de DRF, et la clé `__all__` devient `non_field_errors`.

9.3 Contraintes violées malgré la validation

Deux requêtes envoyées en même temps peuvent toutes les deux passer la validation, puis entrer en conflit au moment d'écrire. Par exemple, deux éditeurs mettent chacun une destination en position 2. La contrainte `itinerary_destination_unique_trip_order` refuse alors la deuxième écriture. Le gestionnaire d'exceptions transforme l'`IntegrityError` en réponse 400. Il retrouve le bon message grâce au nom de la contrainte que PostgreSQL renvoie dans son diagnostic.

Extrait 17 – Message de la contrainte violée (`apps/core/exceptions.py`, extrait).

```
def integrity_error_message(exc):
    """Retrouve le message de la contrainte violée à partir du nom renvoyé par PostgreSQL."""
    diag = getattr(exc.__cause__, "diag", None)  # psycopg expose le diagnostic du serveur
    name = getattr(diag, "constraint_name", None)
    return _constraint_messages().get(name, INTEGRITY_MESSAGE)
```

Le client reçoit donc « Une destination occupe déjà cette position dans le voyage. » avec le code `integrity_error`, au lieu d'une erreur 500.

9.4 Transactions

Les opérations qui écrivent plusieurs lignes sont atomiques. Les services `create_trip`, `clone_trip` et `change_participant_role` ont le décorateur `@transaction.atomic`, comme chaque mutation GraphQL. Le service `add_participant` ouvre un point de sauvegarde (un `transaction.atomic()` imbriqué) autour de l'insertion. Si l'unicité est violée, on attrape l'erreur et la transaction englobante reste utilisable.

9.5 Démonstration du rollback

Quand on clone un voyage, on écrit dans quatre tables : le voyage, le participant, les destinations et les activités. Le test de l'extrait 18 provoque une panne à la toute fin, pendant la copie des activités. Il compare ensuite le contenu des quatre tables avant et après la requête.

Extrait 18 – Panne simulée pendant le clonage (`test_trip_actions.py`, extrait).

```python
def test_failure_midway_gives_500_and_leaves_database_unchanged(
    self, client_for, owner, source, monkeypatch
):
    def snapshot():
        return [
            list(model.objects.order_by("pk").values())
            for model in (Trip, TripParticipant, Destination, Activity)
        ]

    def failing_bulk_create(objs, *args, **kwargs):
        raise RuntimeError("panne simulée pendant la copie des activités")

    before = snapshot()
    monkeypatch.setattr(Activity.objects, "bulk_create", failing_bulk_create)
    client = client_for(owner)
    client.raise_request_exception = False

    response = client.post(clone_url(source))

    assert response.status_code == 500
    assert response.json() == {"error": INTERNAL_ERROR}
    assert snapshot() == before
```

Quand la panne arrive, le nouveau voyage, son participant et ses destinations sont déjà insérés. La transaction annule tout. Le client reçoit une erreur 500 en JSON, sans trace de pile, et la base revient exactement à son état de départ. Nous vérifions le même scénario pour la mutation GraphQL `cloneTrip` (`apps/trips/tests/test_graphql.py`) et pour le service seul (`test_services.py`).

10 Sécurité

10.1 Authentification par JWT

L'API ne garde pas de session. Chaque requête contient un jeton JWT [10] émis par SimpleJWT [9]. Le jeton d'accès expire après 15 minutes, donc un jeton volé ne sert pas longtemps. Le jeton de rafraîchissement dure un jour et ne peut servir qu'une fois. À chaque rafraîchissement, on en reçoit un nouveau et l'ancien est mis sur une liste noire. La déconnexion révoque le jeton envoyé, seulement s'il appartient à l'utilisateur connecté.

Extrait 19 – Paramètres JWT (`config/settings/base.py`).

```
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=15),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=1),
    # Chaque rafraîchissement émet un nouveau refresh et met l'ancien sur liste noire.
    "ROTATE_REFRESH_TOKENS": True,
    "BLACKLIST_AFTER_ROTATION": True,
    "UPDATE_LAST_LOGIN": True,
    "AUTH_HEADER_TYPES": ("Bearer",),
}
```

Les mots de passe doivent passer les quatre validateurs de Django. Ils vérifient la longueur minimale, la ressemblance avec le profil, les mots de passe trop courants et ceux qui ne contiennent que des chiffres.

10.2 Permissions par rôle

Par défaut, DRF refuse toutes les requêtes anonymes (`IsAuthenticated`). Trois permissions objet (`apps/core/permissions.py`) gèrent ensuite les rôles : `IsTripMember`, `IsTripEditor` et `IsTripOwner`. Elles acceptent un voyage ou n'importe quel objet lié à un voyage. Par exemple, une activité remonte à son voyage par `activity.destination.trip`.

TABLE 17 – Accès à un voyage selon le rôle (codes HTTP).

Opération	Propriétaire	Éditeur	Lecteur	Non participant
Lire le voyage, `stats`, `clone`	200 / 201	200 / 201	200 / 201	404
Modifier le voyage, ses destinations, ses activités	200	200	403	404
Supprimer le voyage	204	403	403	404
Gérer les participants	201 / 200 / 204	403	403	404

10.3 Contrôle d'accès au niveau objet

Le risque le plus important pour une API est le mauvais contrôle d'accès au niveau objet (*Broken Object Level Authorization*, API1 de l'OWASP [13]). Il suffit alors de changer l'identifiant dans l'URL pour atteindre l'objet de quelqu'un d'autre. Dans TravelHub, ce cas est évité dès la conception, car chaque vue, chaque resolver et chaque filtre part de `visible_to(user)`. Le voyage d'un autre n'est pas dans le queryset, et la réponse est un 404. Nous appliquons la même idée au parent lors d'une création. Créer une destination sur le voyage de quelqu'un d'autre donne 404, et sur un voyage visible mais sans droit d'écriture, 403.

10.4 Configuration de production

Le module `config.settings.prod` active les protections pour le transport et le navigateur. On y trouve la redirection HTTPS, HSTS sur 30 jours, les cookies `Secure`, `X-Frame-Options: DENY`, `nosniff`, la politique de référent et COOP. L'API navigable de DRF est désactivée, seul le JSON est servi, et GraphiQL et l'introspection sont fermés. Un test lance la commande suivante et vérifie qu'elle ne donne aucun avertissement [3].

Extrait 20 – Vérification de la configuration de production.

```
python manage.py check --deploy --settings=config.settings.prod
```

CORS. Seules les origines de `CORS_ALLOWED_ORIGINS` sont acceptées, et les identifiants ne sont pas transmis. La liste est vide pour l'instant. Au TP2, il faudra y ajouter l'origine du front-end.

Secrets. La clé secrète et l'accès à la base sont lus dans l'environnement. Git ignore le fichier `.env`, et `.env.example` ne contient que des valeurs inventées. Un test vérifie que `SECRET_KEY` vient bien de l'environnement. Les journaux ne contiennent jamais les corps de requête, les en-têtes ou la chaîne de requête. Par sécurité, le formateur masque en plus les champs dont le nom fait penser à un secret, ainsi que les JWT qui apparaîtraient dans un message.

10.5 OWASP API Security Top 10

Le tableau 18 montre ce que nous avons fait pour chacun des dix risques de l'édition 2023.
Chaque ligne a ses propres tests. Le fichier `test_object_level_authorization.py` teste chaque point d'accès REST et chaque opération GraphQL avec un non-participant, un lecteur et un anonyme. Il vérifie qu'ils sont refusés et que la base n'a pas changé après le refus.

TABLE 18 – Mesures prises pour l'OWASP API Security Top 10 (2023).

Risque	**Mesures**
API1 Object Level Authorization	`visible_to()` partout, 404 plutôt que 403, parent cherché parmi les objets visibles
API2 Authentication	Jeton d'accès de 15 min, rotation et liste noire, validateurs de mot de passe, cookie de session ignoré par GraphQL
API3 Object Property Level Authorization	Listes blanches de champs. `owner`, totaux et horodatages en lecture seule. Pas de `is_staff` à l'inscription
API4 Resource Consumption	100 requêtes/h en anonyme et 1000/h par utilisateur, pagination, profondeur GraphQL limitée, délai de 5 s vers l'extérieur
API5 Function Level Authorization	Permissions par rôle, seul le propriétaire gère les participants
API6 Sensitive Business Flows	Inscription, connexion et clonage soumis aux limites de requêtes
API7 Server Side Request Forgery	URL externes constantes, ville en paramètre et code pays en variable GraphQL
API8 Security Misconfiguration	`check -deploy` sans avertissement, en-têtes de sécurité, erreurs JSON sans trace
API9 Inventory Management	Une seule version, décrite par OpenAPI et par la collection Postman
API10 Unsafe Consumption of APIs	Réponses externes validées et converties en `dataclasses`, erreurs 502, 503 et 504

10.6 Risques résiduels

Nous connaissons trois risques que nous avons choisi d'accepter. L'inscription et l'ajout d'un participant disent si un courriel est déjà inscrit. Quelqu'un pourrait donc deviner des courriels, mais les limites de requêtes le ralentissent. Ensuite, GraphQL limite la profondeur des requêtes mais pas leur largeur (avec plusieurs alias). Enfin, la documentation OpenAPI reste publique en production, alors que l'introspection GraphQL y est fermée.

11 Tests

11.1 Stratégie

Nos tests utilisent pytest et pytest-django. Ils tournent sur une vraie base PostgreSQL (`test_travelhub`), créée puis supprimée à chaque exécution. SQLite aurait été plus rapide, mais il ne gère pas l'index unique partiel ni certaines contraintes `CHECK`. Nos tests d'intégrité n'auraient alors rien prouvé. Aucun test ne passe par le réseau. La fixture `mock_http`, basée sur la bibliothèque `responses`, intercepte Open-Meteo et Countries, et elle nous permet de simuler leurs pannes.
Les fixtures communes (`conftest.py`) gardent les tests courts et faciles à lire. Il y a deux utilisateurs, `user_a` et `user_b`, avec leurs objets et leurs clients authentifiés, pour tester les accès croisés. Des fabriques (`make_trip`, `make_destination`...) créent des objets valides dont on peut changer n'importe quel champ. Enfin, la fixture `graphql(query, variables, user=...)` envoie une requête GraphQL avec un vrai jeton.

11.2 Couverture fonctionnelle

11.3 Résultats et couverture

La suite contient **724 tests** et ils passent tous (dernière exécution le 27 octobre 2026, environ trois minutes). La couverture est mesurée sur `apps/` et `config/`, sans les tests et les migrations. Elle est de **98 %** : sur 2 139 instructions et 242 branches, seulement 16 instructions ne sont jamais exécutées. L'intégration continue échoue si la couverture totale descend sous 80 % (`-cov-fail-under=80`). Ce seuil n'est pas un but en soi. Les lignes qui restent sont surtout des cas défensifs.
Le rapport HTML de la figure 7 ne liste que les fichiers qui ne sont pas couverts à 100 %. Les 54 autres sont masqués, c'est pour ça que son total ne compte que 832 instructions.

11.4 Collection Postman

La collection `postman/TravelHub.postman_collection.json` (format v2.1) rejoue un parcours complet sur un vrai serveur. Elle contient 85 requêtes réparties dans neuf

TABLE 19 – Scénarios vérifiés.

Domaine	**Cas**	**Attendu**
Authentification	Sans jeton, jeton falsifié, non signé ou expiré, refresh révoqué	401
Voyages	Création, fin avant début, titre vide, filtres, tri	201 / 400 / 200
Accès croisés	B lit, modifie ou supprime le voyage de A	404
Rôles	Un lecteur modifie, un éditeur supprime ou invite	403
Destinations	Dates hors du voyage, position en double, voyage d'un autre	400 / 404
Activités	Coût négatif, date hors du séjour, filtre par catégorie	400 / 200
Transaction	Panne forcée pendant le clonage	500, base inchangée
Open-Meteo	Ville inconnue, service en 500, délai dépassé	404 / 503 / 504
Countries	Code inconnu, liste `errors` non vide	404 / 502
Agrégation	Une source en panne	200 et `warnings`
GraphQL	Sans jeton, mutation sur le voyage d'un autre, validation, profondeur	`extensions.code`
Sécurité	Affectation de masse, en-têtes, journaux sans secret, `check -deploy`	(divers)

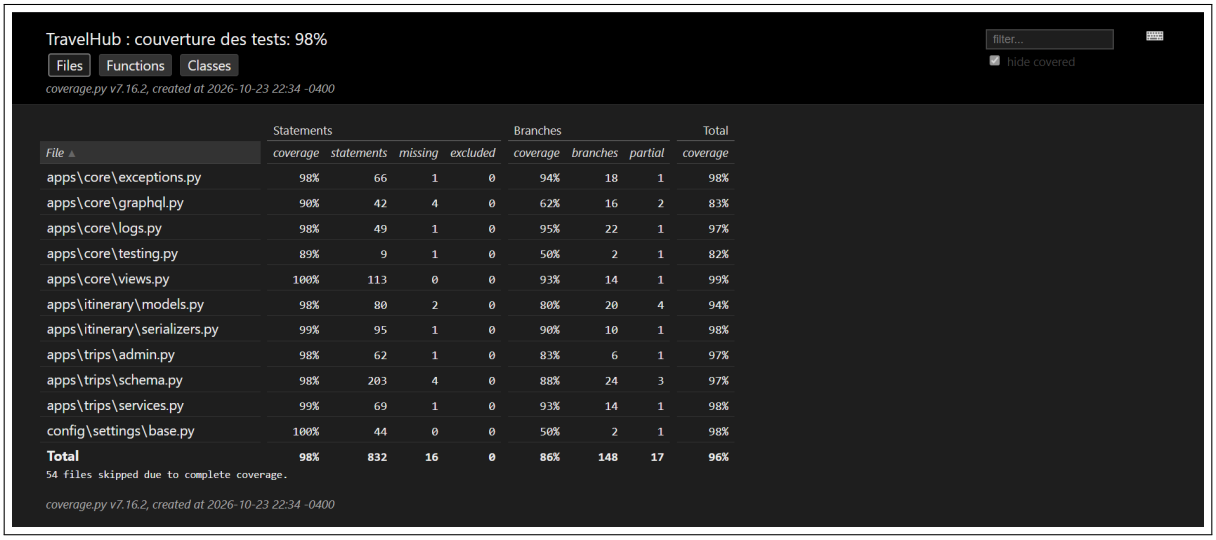


FIGURE 7 – Rapport de couverture HTML (`htmlcov/index.html`). Seuls les fichiers qui ne sont pas entièrement couverts sont affichés.

dossiers (tableau 20). Chaque requête a des assertions sur le statut et le corps de la réponse. Le script de la requête de connexion garde les jetons dans l'environnement, et toutes les requêtes héritent de l'autorisation *Bearer* `{{access}}`.

TABLE 20 – Dossiers de la collection Postman.

Dossier	Requêtes	Contenu
Auth	6	Inscription de deux comptes, connexion, rotation, profil, déconnexion
Trips	9	CRUD, recherche, filtre, tri, statistiques, duplication
Destinations	6	CRUD et filtres
Activities	7	CRUD, filtre par catégorie, statistiques après ajout
Participants	10	Lecteur puis éditeur refusés en écriture (403), puis retrait (404)
Externes	3	Météo, pays, vue d'ensemble
GraphQL	9	`me`, `myTrips`, `trip`, mutations, `destinationOverview`
Cas d'erreur	33	400, 401, 404, 405, 415, affectation de masse, erreurs GraphQL
Nettoyage	2	Suppression des données créées pendant l'exécution

Chaque exécution crée de nouveaux comptes et calcule ses dates à partir du jour même. On peut donc relancer la collection sans rien préparer. Elle se lance dans le *Runner* de Postman ou en ligne de commande avec Postman CLI :

Extrait 21 – Exécution de la collection en ligne de commande.

```
postman collection run postman/TravelHub.postman_collection.json -e
    postman/TravelHub.postman_environment.json
```

La figure 8 montre une exécution avec Postman CLI. Le résumé à la fin indique 85 requêtes, 170 scripts de test et 360 assertions, sans aucun échec.
Les deux outils se complètent bien. Avec pytest, nous simulons les pannes qu'on ne peut pas provoquer quand on veut, comme un service externe en 500, un délai dépassé ou une panne au milieu d'une transaction. Postman, lui, vérifie le comportement de bout en bout, avec le vrai serveur et les vrais services externes.

11.5 Qualité du code

Ruff s'occupe de l'analyse statique (`ruff check`) et du formatage (`ruff format`). L'intégration continue lance ces deux vérifications avant les tests.

12 Industrialisation

Cette partie va plus loin que l'énoncé. Nous l'avons faite seulement après avoir terminé tout ce qui était demandé.

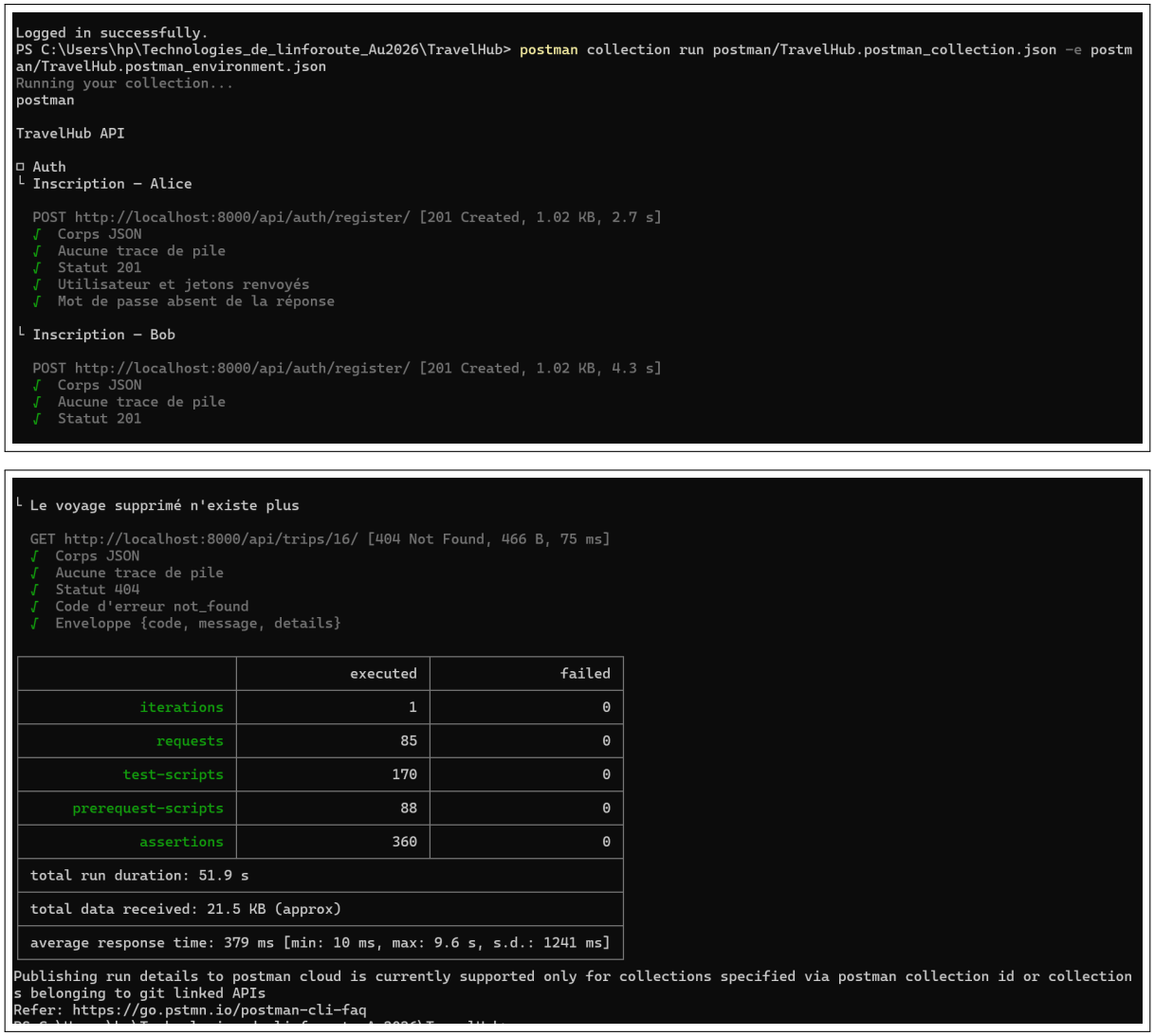


FIGURE 8 – Exécution de la collection avec Postman CLI. En haut, le début de l'exécution avec les premières assertions. En bas, la dernière requête et le résumé (85 requêtes, 360 assertions, 0 échec).

12.1 Docker

Le `Dockerfile` construit l'image en deux étapes. La première part de `python:3.14-slim` et installe les dépendances dans un environnement virtuel. La deuxième copie cet environnement dans une image d'exécution qui n'a pas les outils de construction. L'image lance `collectstatic`, puis WhiteNoise sert les fichiers statiques. Gunicorn tourne avec un utilisateur sans privilèges, qui peut seulement écrire dans `/app/logs`. Une instruction `HEALTHCHECK` appelle `/health/` toutes les 30 secondes.
Le fichier `docker-compose.yml` regroupe trois services (tableau 21).

TABLE 21 – Services Docker Compose.

Service	Rôle
`web`	Image du projet avec les paramètres de production. Applique les migrations puis lance Gunicorn, une fois que `db` est prêt
`db`	`postgres:18` avec un volume persistant et un contrôle `pg_isready`. Le port n'est pas publié
`pgadmin`	pgAdmin 4, accessible seulement depuis le poste local, avec le serveur TravelHub déjà déclaré

12.2 Intégration continue

Le flux GitHub Actions (`.github/workflows/ci.yml`) tourne à chaque *push* sur `master` et à chaque *pull request*. Il a deux tâches.
La première tâche s'appelle « Lint et tests ». Elle démarre un service PostgreSQL 18, puis Ruff vérifie le code et son formatage. Pytest est lancé ensuite avec le seuil de couverture de 80 %. Le résumé de couverture s'affiche sur la page de l'exécution et les rapports sont publiés comme artefacts.
La deuxième tâche construit l'image Docker avec un cache et la démarre avec `docker compose up -wait`. Elle vérifie ensuite que `/health/`, `/api/docs/` et `/api/schema/` répondent.
Dependabot propose aussi chaque semaine les mises à jour des dépendances pip et des actions GitHub. La figure 9 montre l'historique des exécutions. Elles ont toutes réussi, y compris celles lancées par les pull requests de Dependabot.

12.3 Journaux et contrôle de santé

Chaque événement donne une ligne JSON, écrite sur la console et dans un fichier rotatif (10 Mo, 5 archives). L'intergiciel `RequestIdMiddleware` donne un identifiant à chaque requête. On le retrouve dans toutes les lignes de journal de la requête et dans l'en-tête `X-Request-ID` de la réponse. Si un utilisateur signale une erreur, une seule recherche suffit pour la retrouver.

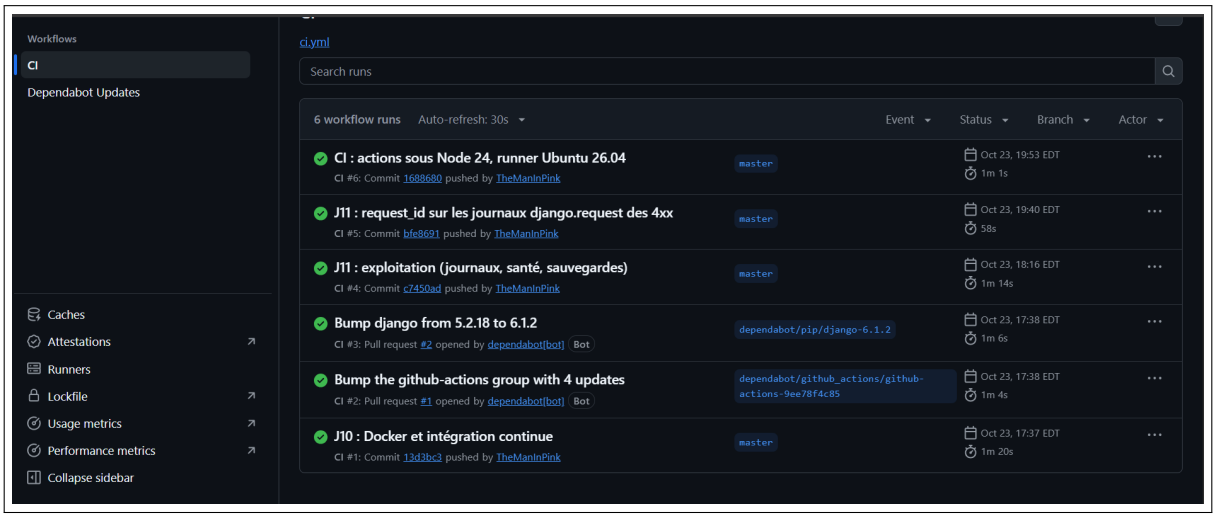


FIGURE 9 – Historique du flux CI sur GitHub Actions. Les six exécutions ont réussi.

Extrait 22 – Ligne de journal d'une requête.

```
{"timestamp": "2026-10-23T22:01:39.510+00:00", "level": "INFO", "logger": "travelhub.request",
 "message": "GET /api/trips/ 200", "request_id": "292f04c8d5e541e69c07c7565e0e65f0",
 "method": "GET", "path": "/api/trips/", "status": 200, "duration_ms": 401.1, "user_id": 5}
```

Le point d'accès `/health/` vérifie la base avec `SELECT 1` et le cache avec une écriture suivie d'une lecture. Il répond 200 si tout va bien et 503 sinon. La cause de l'échec n'est pas donnée au client, mais elle est écrite dans les journaux.

12.4 Sauvegarde et restauration

Les scripts `scripts/backup.ps1` et `scripts/restore.ps1` utilisent `pg_dump` et `pg_restore`. Ils marchent pour la base locale et pour celle du conteneur. Chaque archive est relue avant d'être gardée, et on ne garde que les sept plus récentes. La restauration se fait dans une seule transaction, donc une archive corrompue ne touche pas la base. Une base qui n'est pas vide n'est écrasée qu'avec l'option `-Force`. Pour vérifier une sauvegarde, nous la restaurons dans une base neuve et nous contrôlons le schéma avec `migrate -check`.

13 Conclusion

13.1 Bilan

Le modèle relationnel a des relations 1-N et une relation N-N avec ses propres attributs, et PostgreSQL vérifie les règles importantes. Les mêmes fonctions sont disponibles

dans une API REST documentée avec OpenAPI et dans une API GraphQL. Les deux partagent la logique métier, les permissions et la validation. Les deux services externes sont appelés avec un délai, un nouvel essai, un cache et une dégradation partielle. Pour la sécurité, nous avons suivi l'OWASP API Security Top 10, et 724 tests automatisés couvrent 98 % du code.
Quelques choix nous ont beaucoup aidés. Comme toutes les requêtes partent de `visible_to()`, le contrôle d'accès au niveau objet est réglé une seule fois, et pas vue par vue. Valider à la fois dans l'application et dans la base nous donne des erreurs claires, tout en gardant les données cohérentes quand deux requêtes arrivent en même temps. Enfin, grâce aux services métier partagés, ajouter l'API GraphQL n'a demandé qu'une nouvelle couche de présentation.

13.2 Limites

Notre travail a quand même des limites. En production, nous n'avons pas configuré de moteur de cache. Django utilise donc un cache en mémoire propre à chaque processus. Les réponses externes et les compteurs de limites de requêtes ne sont pas partagés entre les processus Gunicorn. Avec deux processus, la limite réelle est donc deux fois plus haute.
Les prévisions météo ne vont pas au-delà de 16 jours, et Countries donne les noms de pays seulement en anglais. Les risques résiduels de la section 10 sont toujours là (on peut deviner des courriels, la largeur des requêtes GraphQL n'est pas limitée, la documentation OpenAPI est publique). Enfin, la pagination GraphQL par décalage devient lente sur de très grandes listes. Une pagination par curseur serait plus solide.

Références

[1] Trevor Blades. Countries : a public GraphQL API for information about countries, continents, and languages. `https://github.com/trevorblades/countries`, 2026. Point d'accès : `https://countries.trevorblades.com/graphql`. Consulté le 23 octobre 2026. pages 17

[2] Tom Christie and Encode OSS. Django REST framework. `https://www.django-rest-framework.org/`, 2026. Version 3.18. Consulté le 23 octobre 2026. pages 6, 21

[3] Django Software Foundation. Deployment checklist. `https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/`, 2025. Consulté le 23 octobre 2026. pages 35

[4] Django Software Foundation. Django 5.2 documentation. `https://docs.djangoproject.com/en/5.2/`, 2025. Consulté le 23 octobre 2026. pages 6

[5] drf-spectacular contributors. drf-spectacular : Sane and flexible OpenAPI 3 schema generation for Django REST framework. `https://drf-spectacular.readthedocs.io/`, 2026. Version 0.30. Consulté le 23 octobre 2026. pages 6, 26

[6] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. pages 29

[7] Graphene contributors. Graphene-django documentation. `https://docs.graphene-python.org/projects/django/en/latest/`, 2025. Version 3.2. Consulté le 23 octobre 2026. pages 6, 26

[8] GraphQL Foundation. GraphQL specification, October 2021 edition. `https://spec.graphql.org/October2021/`, 2021. Consulté le 23 octobre 2026. pages 26

[9] Jazzband. Simple JWT documentation. `https://django-rest-framework-simplejwt.readthedocs.io/`, 2025. Version 5.5. Consulté le 23 octobre 2026. pages 6, 34

[10] Michael B. Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). RFC 7519, Internet Engineering Task Force, 2015. `https://www.rfc-editor.org/rfc/rfc7519`. pages 34

[11] Open-Meteo. Geocoding API. `https://open-meteo.com/en/docs/geocoding-api`, 2026. Consulté le 23 octobre 2026. pages 17

[12] Open-Meteo. Weather forecast API. `https://open-meteo.com/en/docs`, 2026. Consulté le 23 octobre 2026. pages 17

[13] OWASP Foundation. OWASP API security top 10 – 2023. https://owasp.org/API-Security/editions/2023/en/0x11-t10/, 2023. Consulté le 23 octobre 2026. pages 5, 35

[14] The PostgreSQL Global Development Group. PostgreSQL 18 documentation. https://www.postgresql.org/docs/18/, 2025. Consulté le 23 octobre 2026. pages 6